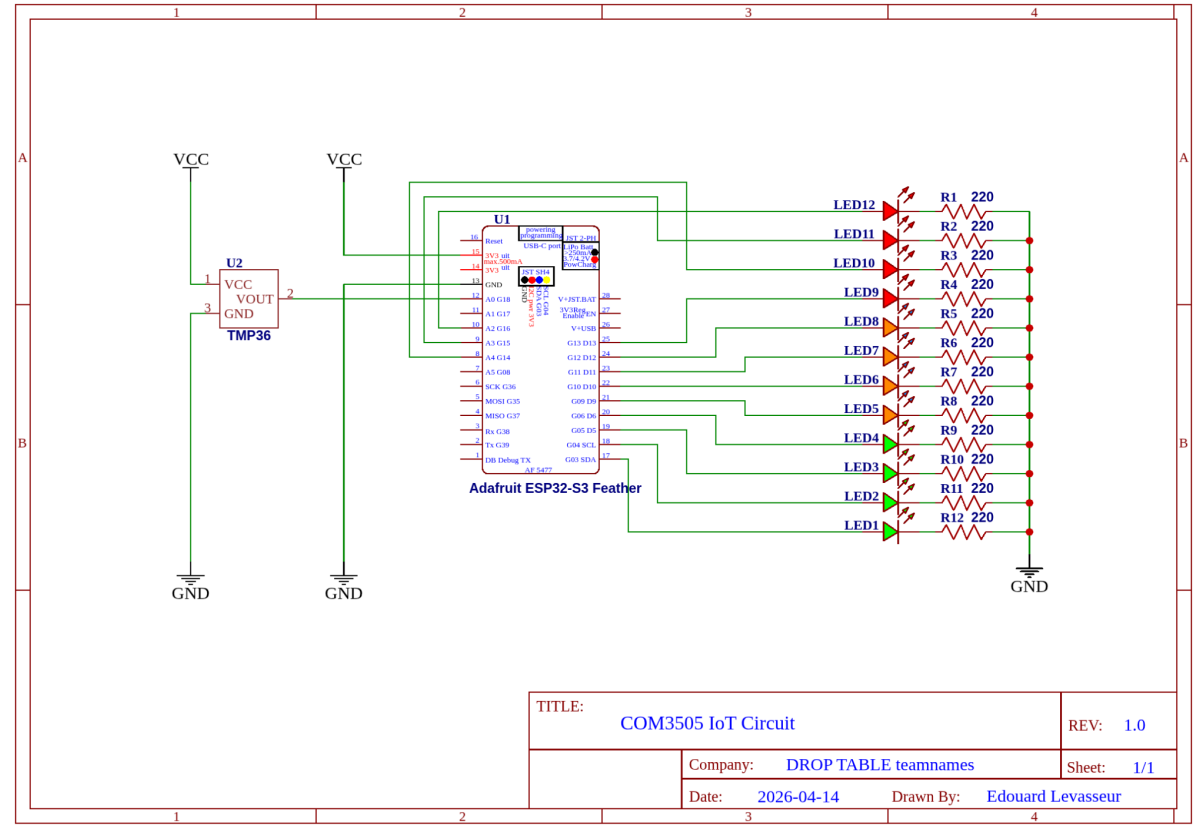
COM3505 Assignment Report

DROP TABLE teamnames
Max Manning and Edouard Levasseur
https://github.com/lewardo/COM3505-IoT

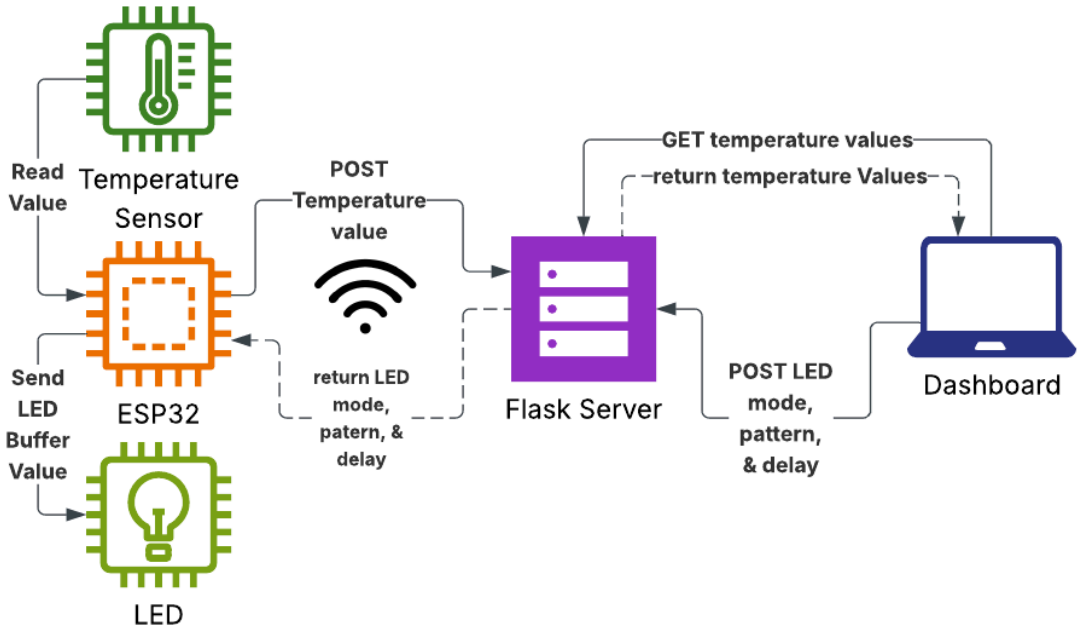
Hardware Setup Diagram



Drawn by Edouard Levasseur using EasyEDA Standard

The circuit can be assembled on a breadboard using jumper cables, as shown above. The LED pin numbers and temperature sensor input can be easily changed in the firmware in LEDs.h and Sensor.h, so the exact wiring is not particularly important. Note that each LED has its own resistor. This isn't mandatory, since the ESP32 is a 3.3V device and the LEDs can conceivably take this, but a small resistor reduces the current draw from the pins, which is better practice.

System Architecture Diagram



Drawn by Max Manning using LucidChart

The Flask server runs on a laptop and serves both a frontend and responds to POST requests to update internal state, either from the ESP station or from a client.

The ESP station periodically sends updates to the server with the current temperature that it reads from the sensor, and receives the LED patterns it should start displaying.

When the server receives an update from the station with the latest temperature, it updates its current temperature array and responds with the current LED state and mode so the ESP can update its patterns.

When the frontend is requested, a webpage is returned to the client to show the dashboard. This frontend constantly polls for changes, which allows the client-side graph to keep updating with the new temperatures.

When new LED parameters are selected on the dashboard, this update is POSTed to the server, which updates the internal state configuration to be sent to the ESP station next time it polls the server.

Client Architecture and Justification

The firmware architecture was designed keeping the limitations of embedded systems in mind. The Arduino IDE and ecosystem were used, as they were the simplest to set up, though not the industry standard (which would be custom toolchains or PlatformIO).

All setup and initialisation is in the setup() function, which provisions network credentials, initialises libraries, waits until a WiFi connection, then starts the network callback timer. Note, the WiFi event loop runs on core 1 in the ESP32 Arduino core, but network-related calls are still blocking for the function that calls them - having the network logic be called asynchronously from a callback function that runs on either core means that no visible artefacts with the LED timings are caused when the network methods are called. The main loop() manages the LEDs - rather than having delay-based timing (which can drift over time), the timings are based on the millis() timer, which cannot drift (being a hardware timer running off the CPU clock), so the patterns stay synchronised.

The patterns are based on an interval (period of the animation) and progress (into the animation), and the custom LED driver manages the relevant LEDs.

Hardcoding network credentials in the firmware would be inconvenient and rigid, meaning that switching networks would require a rebuild and re-upload, which was not optimal when changing location while developing the firmware. Not having to provide credentials every time the microcontroller resets is also convenient. Therefore, we wrote a system that allows you to provide network credentials and a server IP address over the serial connection at bootup, which saves them to persistent storage if they are valid. This way, if the provisioning times out, the previously provided credentials are fetched from storage and used. If no records are in persistent storage and none are provisioned, a default fallback applies, so all cases are covered.

There are three custom driver objects we wrote for this project:

1. The AnalogSensor class is the simplest, abstracting away a few calls to analogue-related setup and reading conversion.
2. The Easings library is a convenience to convert from animation progress to the number of LEDs that should be lit. The class stores a maximum progress value and number of LEDs, the relevant easing function is used to interpolate how many of the indicators should turn on when invoked. Currently, only exponential easing is used, as it is the most evident for demonstration.
3. The final and most complex driver is the LEDController class, which manages the setup and writing to the LEDs. It stores LED pin numbers and states, and provides functions for setting, resetting, and writing to individual LEDs. You can also do the same to all the LEDs simultaneously, as well as display gauges, highlight groups, and random LEDs.

All of these were implemented to make the main source file more legible and maintainable, as well as compartmentalising each part of the logic. Modern compilers are smart enough to abstract away the class overheads, given that these are singleton classes (single instantiation). Namespaces could have been used instead to achieve the same effect, but using classes for libraries is more in line with Arduino convention.

The LEDController class was lightly optimised, since it is used so frequently. It was optimised for memory (though the saves are minimal compared to the overhead of implementing the network stack). LED pin numbers are stored as uint8_t rather than int (which defaults to int32_t), reducing the pin array to one quarter of its original size. The LED states are stored bitwise in a single uint32_t, rather than in a boolean array (which does not optimise for size, each bool is a whole byte). This also allows for more efficient setting and writing of the states, as bitwise operations are strongly supported on the Tensilica architecture. Loops were avoided where possible, as they would introduce overhead where block writes are possible. Conditionals were categorically not used in the loop bodies to avoid introducing branch penalties - deterministic sequential alternatives were used for the bitwise operations to optimise the implementation. The compiler would likely have introduced these techniques at any optimisation level more aggressive than -O1, but implementing them manually guarantees they are applied.

Server Architecture and Justification

The system uses a polling-based approach for communication between the ESP device and the server. This decision was primarily driven by simplicity and reliability. Polling is straightforward to implement on both the microcontroller and server side, requiring only standard HTTP requests rather than persistent connections or more complex protocols such as WebSockets or MQTT. Additionally, polling reduces the need to maintain a connection state on the server, simplifying server-side logic and making debugging easier. For an IoT system of this scale, where data is transmitted at regular intervals rather than requiring real-time streaming, polling provides a sufficient and predictable solution without introducing unnecessary architectural complexity.

The server implements comprehensive error handling to ensure robustness and reliability. All incoming requests are fully validated, including checking for required parameters, correct data types, and acceptable value ranges. If a request fails validation, the server returns clear and specific error messages. This approach improves system stability by preventing malformed or malicious data from being processed. It also aids debugging and development, as issues can be quickly identified through meaningful feedback. In an IoT context, where devices may send inconsistent or noisy data, strong validation is essential to maintain data integrity.

The server and dashboard are designed using a modular architecture, where different components (such as data processing and UI elements) are separated into independent units. This design allows the system to be easily extended in the future. For example, new sensors can be added without requiring major changes to existing code, as new modules can be introduced to handle additional data types. This improves maintainability and scalability, ensuring the system remains adaptable as requirements evolve.

The dashboard uses a responsive design, meaning it is optimised for smaller screens as well as larger devices. This ensures that the system is accessible across a wide range of devices, including smartphones, tablets, and desktops. In IoT applications, where users may need to monitor data on the go, this flexibility is particularly important. It also encourages a clean and efficient interface design, prioritising essential information and reducing unnecessary complexity.

LED Patterns

We implemented the following LED patterns:

Pattern	Code	Description
Blink	b	All LEDs alternate between ON and OFF together
Twinkle	w	Neighbouring LEDs blink oppositely
Chase - Linear	l	LEDs light up in order at a constant rate
Chase - Ease In	i	LEDs light up in order at an increasing speed
Chase - Ease Out	o	LEDs light up in order at a decreasing speed
Chase - Ease In & Out	x	LEDs light up in order at a smoothed rate
Rainbow	r	Cycles through the 3 LED colour groups
Flame	f	LEDs turn ON/OFF randomly
Binary	c	LEDs represent an increasing binary value
Thermometer	t	LEDs light up depending on temperature thresholds
Manual	m	User controls each LED individually